



Today's agenda

↳ overriding

↳ Interfaces

↳ Abstract classes

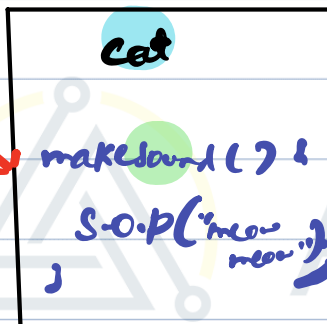
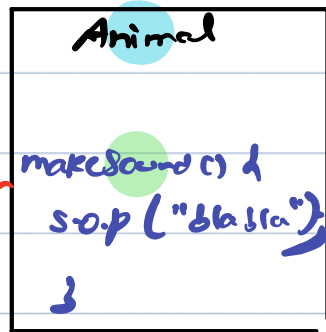
↳ Static



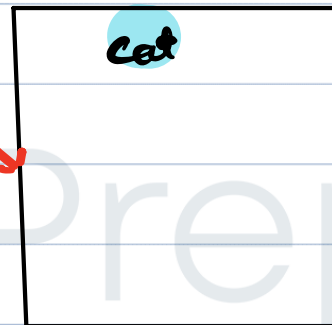
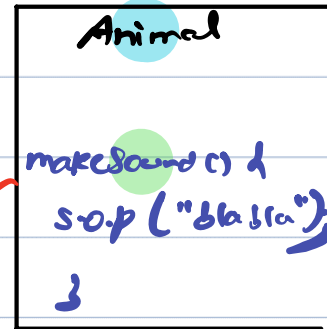
AlgoPrep



* **method overriding** → **overwrite / replace / change**



overriding



overriding

```
Cat obj1 = new Cat();  
obj1.makeSound();  
meow  
meow
```

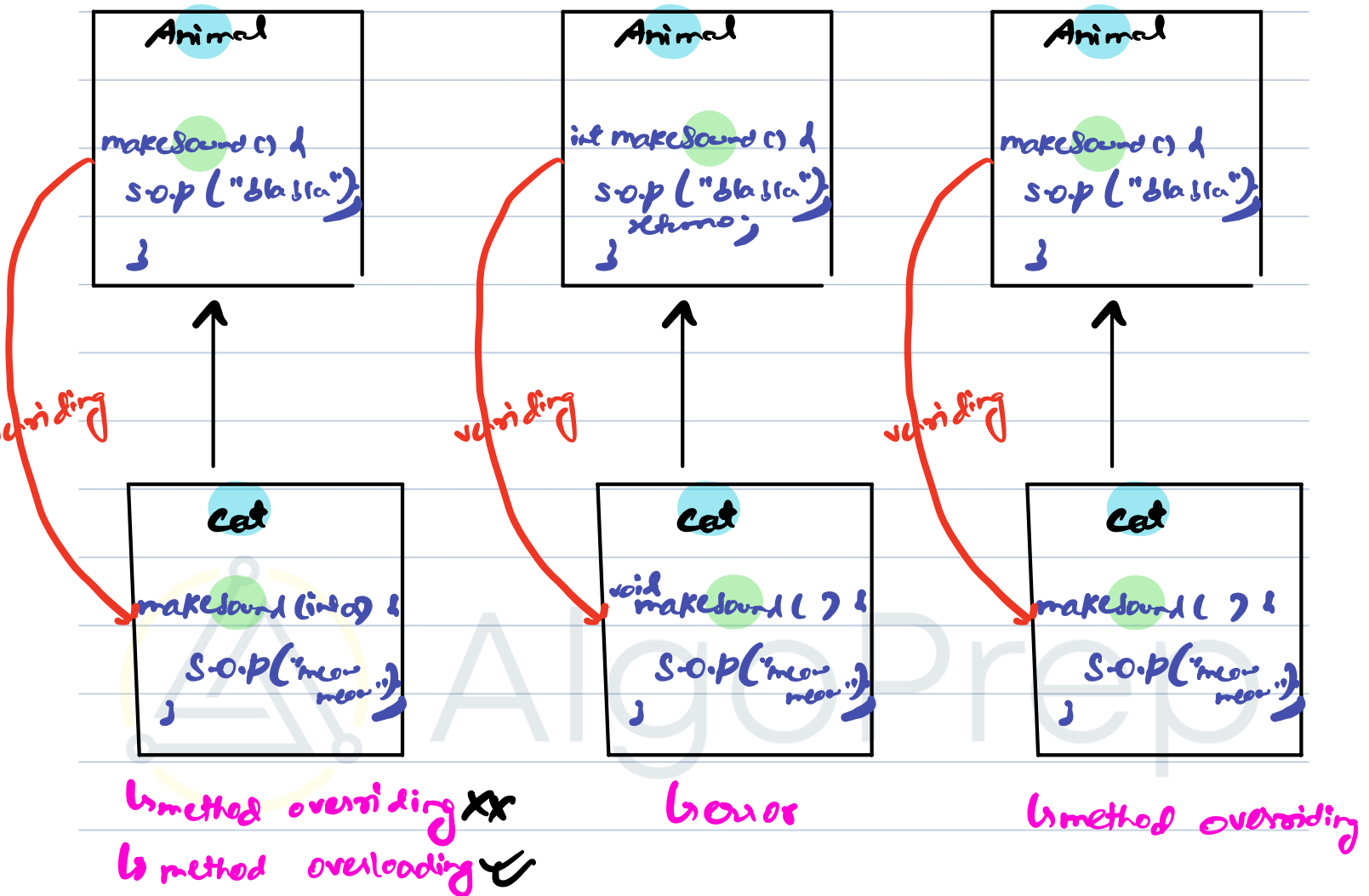
```
Cat Obj2 = new Cat();  
obj2.makeSound();  
bla bla
```

Overriding:

↳ A method with same signature and same return type is present in Parent class as well as child class then method of child class overrides the method of Parent class.



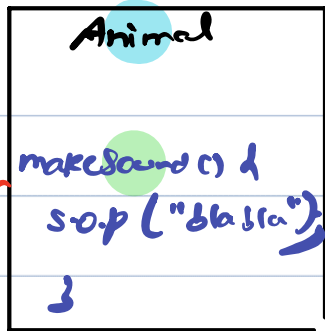
Ex:



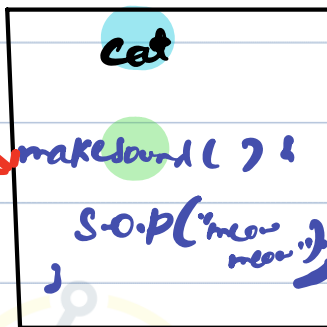
Cat obj1 : new Cat();

obj1.makeSound();

obj1.makeSound(10);



overriding



```
Animal a = new Animal();
a.makeSound(); → bla bla
```

```
Cat c = new Cat();
c.makeSound(); → meow meow
```

```
Cat c = new Animal();
c.makeSound(); → error
```

```
Animal A = new Cat();
A.makeSound(); → meow meow
```

↳ the method that gets pointed is of the object that is stored in the variable and not the data type of variable.



Animal A: new Cat();

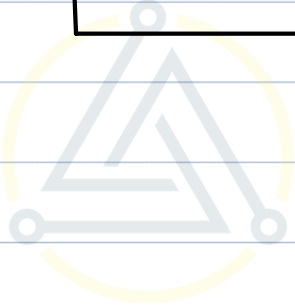
A.makesound(); → Mos



Cat

overrides
makesound() {

 s.o.p("meow
meow")
}



AlgoPrep



Q.

```
class A {  
    void Print() {  
        S.O.P("Hello");  
    }  
}
```

A obj1 = new D();
obj1.Print();
↳ Hey

```
class B {  
  
}
```

A obj2 = new B();
obj2.Print();
↳ Hello

```
class C {  
    void Print() {  
        S.O.P("hey ");  
    }  
}
```

B obj3 = new D();
obj3.Print();
↳ Hey

```
class D {  
  
}
```



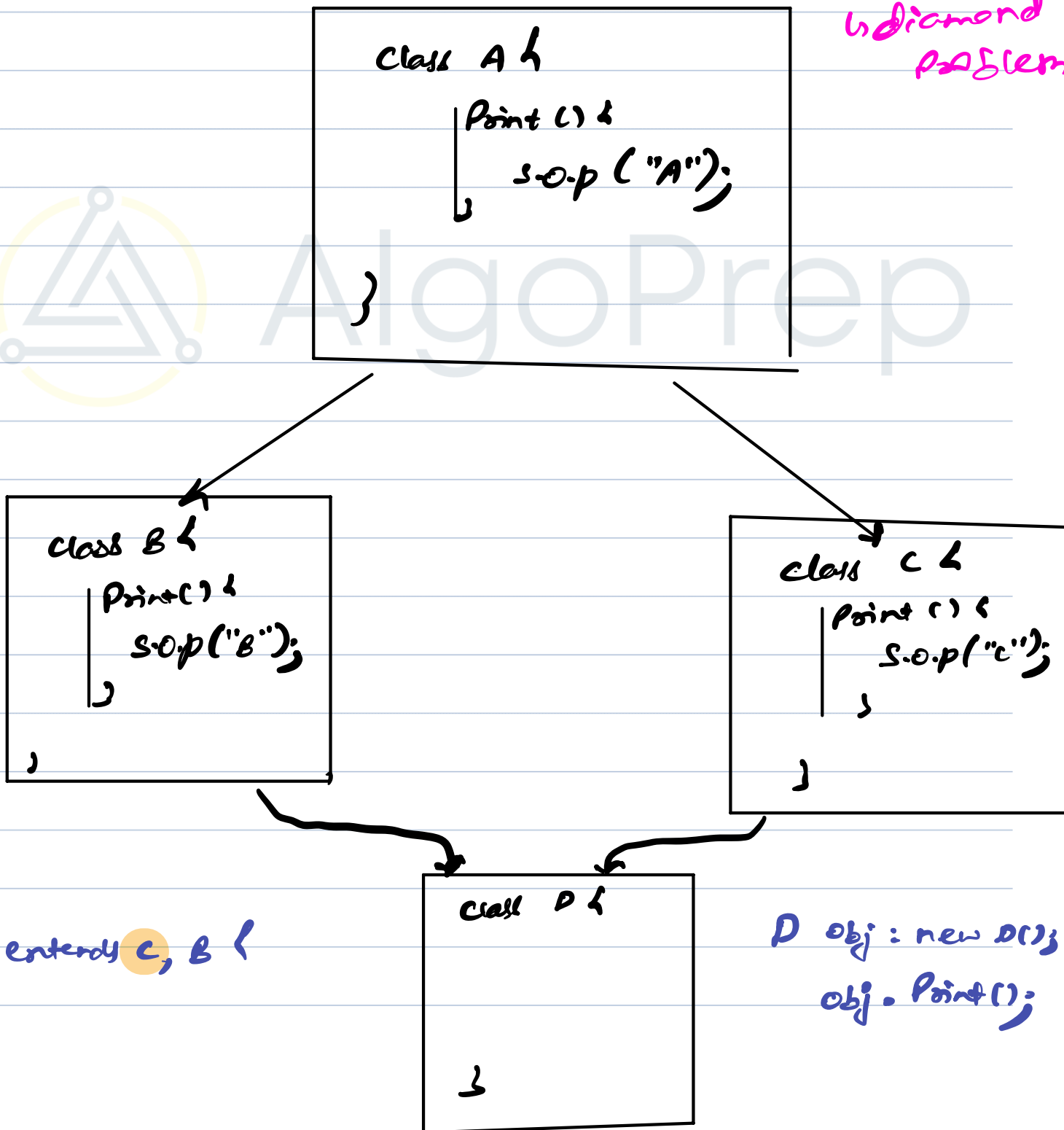
* Multilevel inheritance.

↳ yes, supported in java

* multiple inheritance

↳ no, not supported in java.

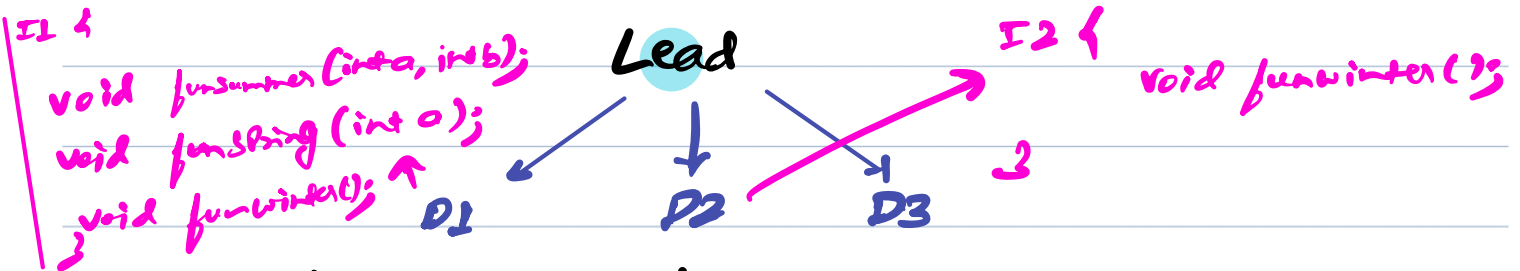
↳ diamond problem





Interfaces

↳ contract / blueprint of behaviours.



funsummer()
funsummer()
funsummer()

funwinter()

↓
class S implements I1, I2 {

void funsummer(int a, int b) {
...
}

void funspring(int a) {
...
}

void funwinter() {
...
}

→ Can't create object of interface directly.

→ One class can implement any no. of interfaces.

→ Interface is just a contract.

↳ implementation of methods will be when you "implements" the interface in a class.



ex:

```
class Pair implements Comparable<Pair> {  
    int compareTo () {  
        3  
    }  
}
```

```
Pair[] arr;  
Arrays.sort(arr);
```

Note: In Interfaces, multi-level inheritance as well as multiple inheritance is possible.



abstract

↳ Partial interface

```
abstract class animal {
```

```
    String name;
```

```
    int age;
```

```
    void makeSound() {
```

```
        s.o.p ("blabla");
```

```
    }
```

```
    abstract void eat();
```

```
}
```

↳ objects can't be created.

```
abstract class dog extends animal {
```

```
    void test() {
```

```
    }
```

```
}
```

↳ inheritance of abstract class is possible
but the child class is also going to be abstract



Abs class

also + methods

inheritance is possible
within class.

Interfaces

methods

implements in class.



AlgoPrep



Static

↳ Creating method or attributes directly for a class rather than its objects.

Ex: `math.abs()`, `math.max()`

```
class math {  
    static int abs(int a) {  
        ;  
    }  
    static int max(int a, int b) {  
        ;  
    }  
}
```

↳ Can you access non-static from static method.

↓
no

↳ Can you access static from non-static method

↓
yes



int[] arr = {19, 20, 20};
↓ ~~bubble sort~~ Quick sort
Arrays.sort(arr);

if (arr[j] > arr[j+1]) {
 swap();

{ 10 20 40 20 }
if (continue to)

int[] arr;
Pair[] arr;

{ (1, 1) (2, 1) (4, 1) (5, 1) }
#ref 1 #ref 2

Arrays.sort(arr);

AlgoPrep